

Writer's Studio (Anti Note-Taking App)

A distraction-free, hierarchical writing environment designed for long-form content creation. Unlike traditional note-taking apps, Writer's Studio focuses on structured document management, seamless export workflows, and deep focus tools.

Key Features

Binder & Organization

- **Hierarchical Structure:** Organize content into infinite nested Folders and Text documents.
- **Drag & Drop:** Easily reorder documents and restructure your project via the Binder.
- **Document Duplication:** Quickly clone existing documents or entire folder structures to iterate on different versions.
- **Search & Replace:** Global search and batch replace functionality across the entire project.
- **Custom Icons:** Personalize folders and documents with unique icons.

Writing Experience

- **Distraction-Free Editor:** Clean interface with focus mode.
- **Teleprompter Mode:** Adjustable scrolling text for reading or presenting.

- **Inspector:** Manage metadata like Status (Draft/Revised/Final), Synopsis, and Research Notes side-by-side with your content. Upload images (for EPUB covers) and audio files (for blog posts) directly to the inspector.

- **History Manager:** Robust Undo/Redo stack to protect your work.

- **Auto-Save:** Changes are automatically persisted to the local SQLite database.

- **Dark Mode:** Toggleable dark/light theme for comfortable writing day or night.

Import & Export

- **Scrivener Import:** Import entire . SCRIV projects (via . SCRIVX file). Automatically parses Binder structure, synopses, and notes.

- **EPUB Support:**
 - **Export:** Generate production-ready EPUBs directly from your project tree.
 - **Open/Reader:** Built-in EPUB reader to view external files.

- **Project Backup:** Full database Export/Restore as a JSON backup, or export the entire project as a single Markdown file.

- **Audio Integration:** Upload MP3/WAV files to documents. Audio is automatically managed during the publishing process and embedded via a custom player.

Interactive Content & Apps

- **Mini-Games:** Built-in interactive experiences like *Centipede*, *Mau Mau*, and *Meridian Mystery Mill*.
- **Browser Extensions:** Support for “Talk” and other browser-based tools integrated into the platform.
- **App Directory:** Access specialized utilities like the *Schnick-Schnack* dictation app directly from the studio.

Integrated Agent

- Built-in chat interface for AI assistance while writing.
- **Voice Interaction:** The chat agent can speak with adjustable speed and a large selection of male and female AI voices using the fast Kokoro engine. FastKoko is a text-to-speech (TTS) engine built on the Kokoro-82M model. This specific model is highly popular in the open-source community because it is open-weight and licensed under the Apache 2.0 license, which allows for both personal and commercial use. Regarding the “David” voice specifically, here is the breakdown of its origin and legal status: Is the “David” Voice Open Source? Technically, yes, if you are accessing it through FastKoko/Kokoro. The Model: Kokoro-82M

is open source. The Voice Weights: The voice profiles (like David) in Kokoro are distributed as part of the model's weight files. Because the weights are released under the Apache 2.0 license, you can use them freely in your own projects without paying royalties. Why it sounds like TikTok You've noticed a similarity because the "David" voice in Kokoro (often labeled as af_david) was trained to mimic a high-quality, professional narrative style that has become the "industry standard" for social media. However, there is a key distinction: TikTok's "Jessie" or "Joey": TikTok uses proprietary voices owned by ByteDance or licensed from companies like ElevenLabs. Those are not open source. Kokoro's "David": This is a synthetic reconstruction or an open-weight equivalent. While it sounds very similar to the "TikTok style," it is a different technical implementation that was released to the public for free use. Use in Your Projects Because FastKoko uses the Apache 2.0 licensed Kokoro model: Commercial Use: You are generally free to use the audio generated by the David voice in monetized videos (YouTube, TikTok, etc.). No API Fees: Unlike ElevenLabs or OpenAI's TTS, since you are running FastKoko locally or on your own server, you don't have to pay per character to use this specific voice.

- Interactive Mode:** Toggle interactive mode to enable continuous voice conversation.
- Chat Management:** Save your chat sessions for later reference.

Sensory Rotation

- Value:** Mandatory focus on smell, sound, and touch over sight.
- Example:** The vibration of a diesel engine through the soles of boots or the taste of grit in the teeth.
- Explanation:** This prevents "visual-only" writing. It creates a three-dimensional world by forcing the AI to describe the harbor's unique auditory and tactile landscape.

- **Explanation:** This grounds the thriller in reality. The harbor itself is a character that provides constant mechanical resistance to the protagonist's goals.

Pulp Density Protocol

Value: Specific narrative ratios (e.g., 55% Action / 30% Dialogue).

- **Example:** Five sentences of Naomi hotwiring a crane for every two sentences of conversation with Inez.
- **Explanation:** This calibrates the “density” of the story, ensuring the AI prioritizes movement and physical tasks over static dialogue or “head-space” thinking.

The Internal Pulse

Value: Practical, risk-ledger thinking; fear expressed only through physiological facts.

- **Example:** *“Risk: High. Battery: 12%. My jaw set so hard the molars ached.”*
- **Explanation:** This defines Naomi's voice. She doesn't “feel” poetic dread; she calculates the cost of failure and monitors her body's stress responses like a piece of machinery.

Chat Memory: The chat has a short memory.



Story Wizard & Prose Generator

The **Story Wizard** is an AI-powered module designed to generate comprehensive narrative structures, character profiles, and full chapters based on high-level directives. It uses advanced prompting strategies to ensure stylistic consistency and “human-like” imperfections in the generated prose.

The story wizard is a specialized narrative engine designed to transform high-level story outlines into detailed, immersive prose. It leverages RAG (Retrieval-Augmented Generation) via ChromaDB to maintain strict consistency with “Story Bibles”—ensuring characters, style guides, and structural rules are never forgotten during the generation process.



Key Features

- **RAG-Powered Story Bible:** Uses ChromaDB to index and retrieve character profiles, style guides, and world-building rules on the fly.

- **Dual-Stage Generation:**
 - **Beat Architect:** Breaks a scene synopsis into 4 distinct, logical narrative beats.
 - **Prose Expansion:** Expands each beat into ~300+ words of immersive narrative.
- **Provider Agnostic:** Supports OpenAI-compatible APIs and Ollama (legacy and V1 formats) for local or cloud inference.
- **Stateful Memory:** Maintains a rolling context window (last 1,000 characters) to ensure smooth transitions between beats.
- **Constraint Enforcement:** Implements a "Humanization Protocol" to strictly avoid banned words and maintain specific pacing styles.

🔑 Key Capabilities

- **Full Chapter Generation:** Generates beats and full prose for multiple chapters in one run.
- **Deep Characterization:** creates psychological profiles, including internal monologues, voices, and "super powers" (critical narrative mechanisms).
- **Stylistic Control:** Enforces specific writing styles (e.g., sensory rotation, advanced rhetoric, anti-cliche protocols).
- **Humanization Protocol:** Inject imperfections, emotional suppression, and specific vocabulary constraints to avoid "AI-sounding" text.

6

Injury Cooldown

- Value:** Persistent physical penalties that do not heal between scenes.
- **Example:** Naomi takes a rope burn in Chapter 4; in Chapter 5, her grip strength is compromised while she tries to climb a container.
 - **Explanation:** This eliminates the "invincible hero" trope. Actions have lasting physical costs that change how the character must solve future problems.

Anti-Looping Protocol

- Value:** Prohibits repeating plans or recapping internal feelings.
- **Example:** Naomi checks a coordinate once; she does not spend three paragraphs wondering if she's doing the right thing.
 - **Explanation:** This keeps the narrative "scene-forward." Every paragraph is required to provide new information, a new constraint, or a physical move.

Environmental Friction

- **Value:** Infrastructure and weather act as active antagonists.
- **Example:** A slippery algae-covered ramp or a timed security gate becomes a barrier during a chase.

47

Integrated Expertise

Value: Mandatory VHF etiquette (Over, Out, Roger) and maritime technical accuracy.

- **Example:** “*Mayday, Mayday, Mayday. This is Vessel Kestrel. Coordinates 41-24 North. Standing by on 16. Over.*”
- **Explanation:** This creates authority in the prose. It replaces generic “phone-style” dialogue with authentic professional communication used in the harbor industry.

Banned Word List

Value: Specific exclusion of “AI-isms” like *tapestry*, *shimmer*, and *salt air*.

- **Example:** Instead of “the salt air filled her lungs,” the AI writes “the scent of low-tide rot and burnt diesel hit her throat.”
- **Explanation:** This forces the AI out of its training-data ruts. By banning clichés, the model is forced to generate more specific, gritty, and industrial sensory details.



Configuration Sections

The Wizard is configured via a dedicated panel (toggle with the  icon).

1. Project Metadata

- **Model:** Select between local models (e.g., qwen2.5:14b) or cloud models (e.g., gpt-4o).
- **Genre & Tone:** Define the core genre (e.g., “Sci-Fi / Noir”) and pacing style.

2. Structural Directives

- **Pulp Density Protocol:** Controls the density of narrative events vs. introspection.
- **Heat Index Protocol:** Manages the “temperature” of the scene (conflict, romance, tension).
- **Anti-Looping:** Instructions to prevent repetitive phrasing or plot cycles.

3. Humanization Protocol

- **Emotional Suppression:** Directs the AI to be subtle with emotions rather than melodramatic.
- **Banned Words:** A list of abstract nouns or clichés to strictly avoid.

- **Simile Restriction:**Limits the use of “like” or “as” to force stronger metaphorical language.

4. Character Profiles

Define the cast with specific fields: - **Voice:** How they speak. - **Internal Monologue:** Their private thoughts. - **Super Power:** A unique narrative or physical ability that drives the plot.

5. Style Guide

- **Sensory Rotation:** Ensures all five senses are engaged cyclically.
- **Visceral Impact:** Focus on physical sensations of emotions.
- **Environmental Friction:** How characters struggle against their setting.

6. Chapters

Define the skeleton of your story: - **Synopsis:** A brief summary of what happens. - **Stop Condition:** A narrative trigger that ends the chapter (e.g., “Cliffhanger”). - **Beats:** The Wizard will first generate 4 narrative beats based on the synopsis, then write prose for each beat.

Property & Value Reference Guide

Linguistic Meter

Value: Short lines, sharp nouns, verbs that move. Average sentence length 8–14 words.

- **Example:** “*The winch groaned. Naomi kicked the pawl into place. Steel cable hissed under the strain.*”
- **Explanation:** This prevents “AI wordiness” and rhythmic monotony. Short sentences build tension, while the limit on long sentences ensures they are only used to describe complex actions or peak pressure.

The Rule of Rust

Value: Every object shows wear. Any tool described must be used, fail, or become an obstacle.

- **Example:** Naomi notices a frayed nylon line while docking; later, that same line snaps when the tug creates a wake.
- **Explanation:** This enforces “Object Permanence.” If an AI describes a detail for atmosphere, that detail must eventually impact the plot mechanically.

Example: The Salt-Chalk Dispatch Narrative Engine

This document outlines the technical architecture of the **Salt-Chalk Dispatch** prompt. The prompt is designed to override the default “creative writing” tendencies of AI—which often lean toward flowery, abstract, and consequence-free prose—and replace them with a high-fidelity, blue-collar maritime thriller simulation.

How to Use This Engine

This JSON structure acts as a **systemic constraint**. Instead of telling the AI what to write, it tells the AI **how to think** and **what to forbid**. By enforcing strict linguistic meters and procedural expertise, the resulting output feels less like a generated story and more like a technical log of a high-stakes crisis.

Generation Pipeline

The Wizard follows a strict two-phase process for each chapter to ensure consistency and depth. This pipeline is unique because it forces the AI to plan before writing, preventing aimless prose.

1. Beat Generation (Structuring)

The AI analyzes the Chapter Synopsis, Scene Goal, and Stop Condition. It breaks the chapter into **4 distinct narrative beats**. Each beat acts as a mini-outline, guiding the prose generation.

2. Context-Aware Prose Writing

The compiled beats are processed sequentially. For every beat, the AI receives the **last 1,000 characters** of the previously generated text (including text from previous chapters). This “sliding window” context ensures that the AI remembers immediate events, dialogue, and tone, maintaining narrative continuity across the entire project.

Usage

1. Open the Wizard Panel (right-side control).
2. Fill in the fields or load a preset JSON configuration.
3. Click **Run Wizard**.
4. The Wizard will stream progress logs to the status window.

5. Upon completion, a new Folder with the Project Name will be created in the Binder, containing the generated chapters.

? Technology Stack

- **Backend:** Node.js + Express
- **Database:** SQLite3 (writing_app.db) for self-contained, fast, and reliable data storage.
- **Frontend:** Vanilla JavaScript (ES6 Modules) - No complex build steps or heavy frameworks.

- **Styling:** Vanilla CSS with dark/light mode support.
- **chroma db:** For efficient AI memory when writing stories.
- **Kokoro:** For fast AI voice.
- **OpenAI:** For AI chat.
- **n8n:** For workflow automation.
- **Tailscale:** For secure remote access.
- **Playwright:** For web browsing.
- **MCP:** For inter-agent communication.
- **Eleventy:** For static site generation.
- **eleventy epub and pdf generation:** for bulk epub and pdf generation.

Key File	Description
server.js	Main Express server. Handles API routes, database operations, and EPUB generation.
db.js	SQLite database connection and helper methods.

System Config

- GET /api/config
Retrieve public server configuration (e.g., Manager URL).

Story Wizard & AI

- **POST /api/wizard/run**
Start the AI Story Wizard generation process. Responses are streamed (NDJSON).
Body: Full Wizard Configuration JSON.
- **POST /api/wizard/save**
Save a generated story to the `public/stories` directory.
Body: Full Story JSON object (requires `project_metadata.project_name`).
- **GET /api/wizard/files**
List available Wizard configuration templates and saved stories (JSON files) from the `public/stories` directory.
- **DELETE /api/wizard/files/:filename**
Delete a saved story file.
Param: `filename` (e.g., `my_story.json`).
- **POST /api/wizard/generate**
Wizard Architect endpoint to generate a story concept/configuration from a prompt.
Body: `{ model, prompt }`
- **GET /api/config/models**
List available AI models configured in `.modelenv`.
- **ALL /api/manager/***
Proxy endpoint to forward requests to the external MCP Manager Agent.

Key File	Description
<code>public/js/writer_v2.js</code>	Core frontend logic: state management, API calls, and event handling.
<code>public/index.html</code>	Main application entry point and UI layout.

Installation & Setup

Prerequisites

- Node.js (LTS recommended)
- npm
- Python 3.10+
- Conda (Anaconda or Miniconda)

Quick Start

1. Install Dependencies

```
cd anti
npm install
```

2. **Initialize Database** The application uses a local SQLite database (writing_app.db). If it doesn't exist, it will be initialized automatically on the first run, or you can run:

```
node init_db.js
```

3. **Start the Application** Use the provided startup script to launch the entire system (Backend, Frontend, MCP Servers, and Tailscale Proxy).

```
./start_app.sh
```

4. **Access the App** The script will output a secure Tailscale HTTPS URL (e.g., https://machine-name.tailscale.net). Open this URL in your browser. Alternatively, you can access the local version at: <http://localhost:3000>

The start_app.sh script is the central entry point for the Writer's Studio. It orchestrates the startup of all necessary components in the correct order:

1. **MCP File Server:** Starts the sandboxed file server on port 65432.
2. **Manager Agent:** Activates the mcp Conda environment and starts the AI Manager on port 8888.
3. **Eleventy:** Starts the static site generator in the background (port 8080).

The start_app.sh Script

Search & Manipulation

- GET /api/documents/search?q=term
 - POST /api/documents/replace
- Perform a global search and replace across the project.
- Body:* { query: "old", replacement: "new" }

Import, Export & Backup

- GET /api/export
 - POST /api/import
- Export the entire project database as a JSON array (Backup).
- Includes cover images.
- Restore the project from a JSON backup. **Warning: This overwrites all existing data.**
- Body:* [{ id, title, content, cover_image, ... }, ...]

- GET /api/export/epub?rootId=<id>
 - POST /api/documents/import
- Generate and download an EPUB file.
- Optional Query:* rootId to export a specific folder/chapter.
- Import parsed data (e.g., from Scrivener).
- Body:* { projectName, items: [...] }

Document Management

- **GET /api/documents**
Fetch all documents in the project as a flat list.
- **GET /api/documents/:id**
Retrieve details for a single document.
- **POST /api/documents**
Create a new document (Folder or Text).
Body: { parent_id, title, type, status, synopsis, notes, content, icon, cover_image }
- **PUT /api/documents/:id**
Update an existing document.
Body: { title, content, status, synopsis, notes, parent_id, order_index, type, icon, cover_image }
- **DELETE /api/documents/:id**
Delete a single document.
- **POST /api/documents/batch-delete**
Delete multiple documents and their children.
Body: { ids: [1, 2, 3] }
- **POST /api/documents/batch-stats**
Calculate word counts for specific documents.
Body: { ids: [1, 2, 3] }

4. **Tailscale Proxy:** Sets up a secure HTTPS proxy via Tailscale, allowing you to access the app remotely or securely.
5. **Anti App:** Starts the main Note-Taking App (Express/Node server) on port 3000.

Automatic Cleanup: The script handles graceful shutdowns. When you stop the script (Ctrl+C), it automatically: - Stops the Tailscale serve process. - Kills all background processes (Manager, File Server, Eleventy). - Ensures no stale processes are left on port 3000.

Deployment Workflow

To deploy updated code from your local machine to the server, follow these steps:

1. **Sync Files (from your local machine):**

```
./resync_project.sh
```

2. **Connect to the Server:**

```
ssh kai@192.168.1.5
```

3. **Update Dependencies (on the server):**

```
cd ~/anti  
npm install
```

Multi-Agent MCP System

A modular, multi-agent AI system built on the **Model Context Protocol (MCP)**. This system orchestrates multiple specialized agents to handle web browsing, file operations, and external workflows, controlled by a central Manager.

System Architecture

The system consists of a **Manager** that orchestrates several independent MCP Agents:

Component	Filename	Role
Manager	manager.py	The central brain. Orchestrates tasks, connects to other agents, and manages the user interface (CLI or Web).
MCP Server		Manages model registry, API keys, and context windows.
Model Agent	model-agent.py	

- **Database:** SQLite3 (writing_app.db) for self-contained, fast, and reliable data storage.

- **Frontend:** Vanilla JavaScript (ES6 Modules) - No complex build steps or heavy frameworks.

- **Styling:** Vanilla CSS with dark/light mode support.

Key File	Description
server.js	Main Express server. Handles API routes, database operations, and EPUB generation.
eleventy_mcp.py	Handles Markdown generation, image processing, and content transformation.
db.js	SQLite database connection and helper methods.
public/js/writer_v2.js	Core frontend logic: state management, API calls, and event handling.
public/index.html	Main application entry point and UI layout.

API Reference

The backend exposes a RESTful API for document management, AI generation, and system operations.

- **Local Asset Management:** Cover images and audio files are saved directly within the post’s directory in a nested `assets/` folder, with relative paths automatically updated in the frontmatter.
- **Dynamic Frontmatter:**
 - **Automatic Tags:** Merges manual tags with language, category, and format flags (EPUB, Zine, Onesheet).
 - **No YAML Anchors** Produces clean, human-readable YAML without confusing aliases.
- **Content Blocks:**
 - **Auto-TOC:** Inject a Table of Contents container with dynamic title and author information.
 - **Featured Image:** Optional injection of a Liquid-ready figure block for the cover image.
 - **YouTube Support:** Automatic video player injection for the “Video Blog” category.
 - **Audio Player:** Automatic injection of a custom, mobile-responsive audio player via the `` shortcode.
- **HTML to Markdown Transformation:** Automatically converts `<h2>` editor tags to `##` Markdown headers and preserves paragraph integrity.

? Technology Stack

- **Backend:** Node.js + Express

Component	Filename	Role
WebAgent	<code>webagent_mpc.py</code>	MCP Server. Handles web browsing (Playwright), running tests, and transcript summarization.
Agent Flow	<code>agent-flow.py</code>	MCP Server. Triggers external n8n workflows.
File Server	<code>mcp_file_server.py</code>	MCP Server. Provides safe filesystem access (read/write/list/delete) within a jail directory.
Scripts	<code>tube/yt5subsSingle.sh</code>	Helper script for downloading YouTube subtitles.

Getting Started

Prerequisites

- Python 3.10+

- Node.js & npm (for Playwright)
- Conda (optional but recommended)

Installation

1. Install Python Dependencies

```
conda create -n mcp python=3.12
conda activate mcp
pip install -r requirements.txt
# Or manually:
# pip install aiohttp prompt_toolkit fastapi
uvicorn youtube-transcript-api
```

2. Setup Playwright

```
npm playwright install --with-deps
```

3. Configure Models Edit .modelenv to define your available models (Ollama, OpenAI, etc.):

```
{
  "models": {
    "qwen2.5": { "type": "ollama",
      "base_url": "http://localhost:11434/v1/chat/completions",
      "gpt-4o": { "type": "openai",
        "api_key": "sk-...", "base_url": "https://api.openai.com/v1" }
  }
}
```

- **Cover Image:** Add a visual cover image to any document or folder via drag-and-drop in the Inspector.
- **History Manager:** Robust Undo/Redo stack to protect your work.
- **Auto-Save:** Changes are automatically persisted to the local SQLite database.

Import & Export

- **Scrivener Import:** Import entire .scriv projects (via .scrivx file). Automatically parses Binder structure, synopses, and notes.
- **EPUB Support:**
 - **Export:** Generate production-ready EPUBs directly from your project tree. Uses the folder's Cover Image if available.
 - **Open/Reader:** Built-in EPUB reader to view external files.
 - **Project Backup:** Export/Restore the entire database as a JSON backup. Includes all metadata and cover images.

Eleventy Publishing Workflow

- **One-Click Publish:** Seamlessly export documents to your Eleventy site from the Inspector.
- **Smart Directory Routing:** Automatically saves posts to date-based directories (eleventy/site/YYYY/MM/DD/).

Example

```
python mcp_file_server.py --host localhost --port 65432 conda  
activate mcp ./manager.py --serve --host 100.122.130.97 --port  
8888 npm start npx @11ty/eleventy --serve
```



more Features



Binder & Organization

- **Hierarchical Structure:** Organize content into infinite nested Folders and Text documents.
- **Drag & Drop:** Easily reorder documents and restructure your project via the Binder.
- **Search & Replace:** Global search and batch replace functionality across the entire project.



Writing Experience

- **Distraction-Free Editor:** Clean interface with focus mode.
- **Teleprompter Mode:** Adjustable scrolling text for reading or presenting.
- **Inspector:** Manage metadata like Status (Draft/Revised/Final), Synopsis, and Research Notes side-by-side with your content.

Usage

1. Interactive CLI Mode

The primary way to use the system.

```
python manager.py -i
```

Features:

- **Natural Language Interaction:** “Find me a YouTube video about cats and get the subtitles.”
- **Runtime Model Switching:** - /model <name> or /switch <name>: Switch the active model instantly (e.g., /model gpt-4o).
- /list: List all available models in the registry.
- **Model Propagation:** Switching models in the Manager automatically updates all sub-agents (e.g., WebAgent will use the new model for summarization).

2. Single Query Mode

Execute a single task and exit.

```
python manager.py "Summarize the latest  
news on CNN"
```

3. API Server Mode

Run the Manager as a REST API server with a simple web UI.

```
python manager.py --server --host  
0.0.0.0 --port 8000
```

UI: `http://localhost:8000`

• **Endpoint:** `POST /chat {"message": "..."}`

❓ Features & Capabilities



Model Management (model-agent.py)

- Centralized configuration via `.modelenv`.
- Dynamic model listing.
- Hardware-aware context mapping (e.g., mapping `context_length` to `num_ctx`).



Web Browsing (webagent_mpc.py)

- **Navigation:** `browser_navigate`, `browser_click`, `browser_type`.
- **Testing:** Discovers and runs Playwright tests (`.spec.ts`).

- `run_playwright_test`: To re-run the test and verify fixes.
- `browser_navigate / browser_snapshot`: To inspect the live page if needed.
- **Iteration:**
 - The LLM analyzes the failure.
 - It constructs a tool call (e.g., `fs_write_file` to fix a CSS selector).
 - The system executes the tool and returning the result to the LLM.
 - The LLM can then choose to re-run the test to confirm the fix.
- **Constraints:** The loop is capped at a maximum number of turns (default: 10) to prevent infinite loops if the test cannot be fixed.

Summary

This architecture allows the system to treat test failures not as fatal errors, but as tasks to be solved. By combining direct file access with the ability to run and interpret test results, the agent enters a feedback loop that continues until the test passes or the agent determines it cannot be fixed.

2. Agentic Loop & Test Healing

The “Test Healing” capability is a specific workflow designed to autonomously repair broken Playwright test cases. It is implemented primarily within the `AsyncWebAgentService` in `webagent_mpc.py`.

The Healing Process (`smart_heal_test`)

The process follows a Retrieve-Analyze-Act cycle:

1. **Initial Verification:** The agent first runs the target test file using `run_playwright_test`.
 - If the test passes, the process terminates early.
 - If it fails, the agent captures the **Execution Output** (console logs, error messages).
2. **Context Gathering:**
 - The agent reads the content of the failing **Test Code** from the file system.
 - This data (Code + Logs) is packaged into a context dictionary for the LLM.
3. **The Agentic Loop (`_run_agentic_loop`):**
 - **Role:** The LLM is initialized with a `HEALER_PROMPT`, instructing it to act as an expert test engineer.
 - **Tools:** The agent is granted access to:
 - `fs_write_file`: To modify the test code directly.

- **Agentic Capabilities:**
 - **Self-Healing:** Automatically diagnoses and fixes failing tests (`smart_heal_test`).
 - **Test Generation:** Creates new Playwright tests from natural language descriptions (`smart_generate_test`).
 - **Test Planning:** Generates comprehensive test plans by exploring the target site (`smart_plan_test`).
- **YouTube:** Downloads subtitles using a custom script (`tube/yt5subsSingle.sh`).
- **Summarization:** Summarizes transcripts using the currently active LLM.

Workflows (`agent-flow.py`)

- Triggers n8n workflows via webhooks.

Files (`mcp_file_server.py`)

- Sandboxed file operations in `./transcripts`.
 - Read, write, list, and delete files.
-

Configuration



.modelenv

The .modelenv file is the source of truth for your models.

```
{
  "models": {
    "my-model": {
      "type": "ollama",
      "base_url": "...",
      "context_length": 8192
    }
  }
}
```

```
-i, --interactive | Run in interactive mode. |
-m MODEL | Set the initial model (default: gwen2.5). |
--server | Start in API Server mode. |
--host, --port | Configures the API server address. |
Example: | http://100.122.130.97:8888|
```

Manager Arguments

AgentOrchestrator class to manage the lifecycle of sub-agents.

- **Sub-Agents:** Specialized processes such as model-agent (LLM provider), webagent (browser automation), and elevenly-mcp (site management).

- **Protocol: JSON-RPC 2.0.** Messages are serialized as JSON objects with id, method, and params fields.

- **Transport: Stdio Pipes.** The orchestrator spawns sub-agents as child processes. It writes requests to the child's stdin and reads responses from the child's stdout.

- **Asynchrony:** Python's asyncio library is used throughout. AsyncMCPClient handles the non-blocking I/O, ensuring the orchestrator remains responsive while waiting for agent tools to complete.

Communication Flow

1. **Initialization:** The orchestrator launches a sub-agent process.
2. **Tool Call:** When the orchestrator needs a specific capability (e.g., specific model details), it constructs a JSON-RPC request (e.g., `tools/call`) and writes it to the agent's input pipe.
3. **Execution:** The agent's event loop reads the request, routes it to the appropriate internal function (e.g., `list_models`), and awaits the result.
4. **Response:** The agent writes the result back as a JSON-RPC response to its output pipe, which the orchestrator reads and processes.

File	Location	Purpose
.flowenv	Root	n8n Config. Defines n8n API keys, base URLs, and the registry of executable workflows.
.env	anti/.env	Web App Config. Configures the Express server port, database credentials, and connection to the Manager Agent.
synchronize.env	anti/plugin/	Sync Secrets. Contains credentials (e.g., REMOTE_PASS) for the synchronize_project script.

1. Agent Communication Technology

The system employs a **Hub-and-Spoke** architecture where a central orchestrator manages multiple specialized agents. The communication backbone relies on **JSON-RPC 2.0 over Standard Input/Output (Stdio)**.

Key Components

- **Orchestrator** (manager.py): The central hub that initializes and coordinates all other agents. It uses the



Audio Transcription Feature

The system includes a local audio transcription capability using MLX-Whisper, allowing you to transcribe audio files without relying on external services.

Setup

1. **Install MLX-Whisper** (in your conda environment):

```
/opt/anaconda3/envs/mcp/bin/python -m pip
install mlx-whisper
```

2. **Place audio files** in the transcripts/ directory (the file server is jailed to this location for security).

Usage

Via Manager (Recommended)

In interactive mode, simply ask the manager to transcribe:

```
python manager.py -i
>>> transcribe audio document myfile.mp3
```

The Manager will: 1. Use fs_find_file to locate the audio file in transcripts/ 2. Call transcribe_audio to process it with MLX-Whisper 3. Return the transcribed text

Direct CLI Testing

You can also test the transcriber directly:

```
/opt/anaconda3/envs/mcp/bin/python
transcriber.py -p transcripts/
audio.mp3
```

How It Works

- **File Server** (mcp_file_server.py): Provides `fs_find_file(filename)` to recursively search for audio files within the `transcripts/` jail.
- **Transcriber** (transcriber.py): MCP server that exposes `transcribe_audio(file_path)` using the `mlx-community/whisper-turbo` model.
- **Manager** (manager.py): Orchestrates the workflow by chaining file search → transcription.

 Project File Reference

File	Type	Description
manager.py	Manager	

Email (IMAP) |
list_mailboxes | List all available email folders. |
fetch_email | Download recent headers from a mailbox.
|
read_email | Retrieve the full body of a specific email.
move_email | Archive or move emails between folders. |
flag_spam | Mark a specific sender or message as spam.
Brave Browser |
read_browser_tabs_brave | List all currently open tabs in Brave. |
read_browser_history_brave | Search through Brave's local history. |
Publishing & Deployment |
synchronize_project | Sync project files to remote server via SSH. |
generate_stories_dashboard | Generate dashboard statistics for all projects. |
build_jeff / push_jeff | Specialized build and deployment scripts for the Jeff Meridian site. |
make_epub_jeff | Create production-ready EPUB from project. |
make_zines_jeff | Create formatted Zines for print. |
make_one_sheet_jeff | Create a One-Sheet marketing summary. |

 Environment Configuration

The system uses several environment files to configure different components.

File	Location	Purpose
.modelenv	Root	Model Registry. Defines available LLMs (Ollama, OpenAI, usage), API keys, and context window sizes.

Tool	Description
<code>get_youtube_subtitles</code>	Download subtitles from a YouTube video.
<code>fs_search_transcripts</code>	Search through transcribed text files.
Eleventy / Site	
<code>create_post</code>	Create a new markdown post with frontmatter.
<code>list_all_posts</code>	List all posts in the content directory.
<code>remove_post</code>	Delete a post.
<code>publish_site</code>	Trigger an 11ty build.
n8n Workflow	
<code>n8n_list_workflows</code>	List available workflows in <code>.flowenv</code> .
<code>n8n_trigger_workflow</code>	Trigger a workflow by ID.
<code>n8n_check_executions</code>	Check recent execution logs.
Writer Tools (anti/plugin)	
<code>read_active_document</code>	Read the currently open document in Writer's Studio.
<code>edit_active_document</code>	Modify the content of the active document.
<code>open_document</code> Open a specific document in the editor by ID or name.	
<code>prose_stats</code> Analyze document prose (word count, banned words, readability).	
<code>restart_speech</code> Restart the Kokoro speech engine.	
<code>generate_qr_code</code> Generate a QR code from a string (useful for sharing links).	

File	Type	Description
<code>model-agent.py</code>	MCP Server	Central orchestrator. Manages ag lifecycle, routes LLM requests, h interactive CLI (<code>-i</code>), single quer API server mode (<code>--server</code>). Supports runtime model switchin
<code>webagent_mpc.py</code>	MCP Server	Model registry manager. Loads <code>.modelenv</code> , provides <code>list_m</code> and <code>get_model_config</code> tools. Handles API keys and context wi mappings.
<code>agent-flow.py</code>	MCP Server	Web automation agent. Provides Playwright browser control, runs <code>.spec.ts</code> tests, downloads You subtitles, and summarizes transcr using the active LLM.
<code>mcp_file_server.py</code>	MCP Server	Workflow automation. Triggers e n8n workflows via webhooks usi <code>trigger_flow</code> tool.
<code>transcriber.py</code>	MCP Server	Sandboxed file operations. Provid <code>fs_read_file</code> , <code>fs_write_</code> , <code>fs_list_directory</code> , <code>fs_delete_file</code> , <code>fs_move_file</code> , <code>fs_search_transcripts</code> , <code>fs_find_file</code> . Jailed to <code>./transcripts</code> directory. Confi via <code>.fileserver</code> env file and <code>host/--port</code> CLI args.
		Audio transcription service. Uses Whisper (<code>mlx-community/whisper-turbo</code>) to transcribe files. Provides <code>transcribe_audio(file_</code> tool. Supports CLI mode with <code>-p</code> for testing.

Configuration Files

File	Purpose
.modelenv	Model registry (JSON). Defines available LLMs with type, base_url, API keys, and context lengths.
.filesrvr	File server configuration. Sets HOST and PORT for standalone socket mode.
anti/.env	Web UI configuration. Contains database credentials, server IP (IP), and manager connection details (MANAGER_HOST, MANAGER_PORT).
Path	Description
tube/ yt5subssingle.sh	Bash script for downloading YouTube subtitles. Called by webagent_mpc.py.
transcripts/	Jail directory for file server. All file operations are restricted to this folder. Audio files for transcription must be placed here.
tests/	Playwright test files (.spec.ts). Discovered and executed by webagent_mpc.py.
anti/	Web UI application (Node.js/Express). Provides a browser-based

24

Tool	Description
browser_run_code	Execute raw Playwright code.
browser_console_messages	Retrieve console logs (info/error).

Test Automation

discover_tests	Find all .spec.ts Playwright tests.
run_playwright_test	Run a specific test file.
run_test_by_name	Run a specific test case by name.
smart_heal_test	Agentic: Diagnose and fix a failing test using the LLM.
smart_generate_test	Agentic: Generate a new test from a natural language description.
smart_plan_test	Agentic: Explore a site and plan a comprehensive test suite.

Filesystem (Sandboxed)

fs_list_directory	List files in the sandbox.
fs_read_file	Read file content.
fs_write_file	Write content to a file.
fs_delete_file	Delete a file.
fs_move_file	Move/Rename a file.
fs_find_file	Find a file by name.

Media & Content

transcribe_audio	Transcribe audio files using local MLX-Whisper.
------------------	---

29

(self-fulfilling) mocks. If a plugin’s argument schema changes or if the JSON-RPC implementation breaks, this test suite will fail, providing high confidence in the stability of the Multi-Agent API without risking user data.

Tool Registry

The following tools are available to the Manager and its sub-agents. Custom scripts are located in `anti/plugin` and are automatically discovered.

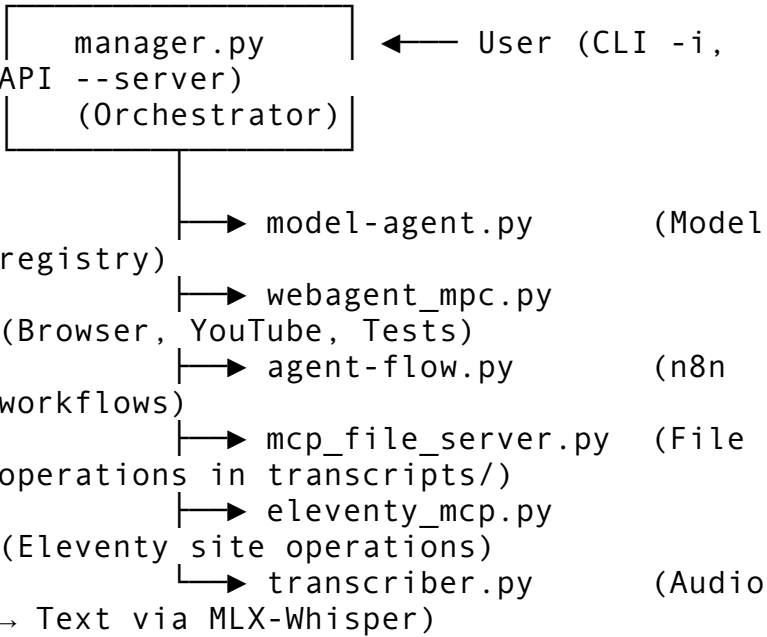
Tool	Description
Model & System	
<code>list_models</code>	List all available models in <code>.modelenv</code> .
<code>get_model_details</code>	Get URL, key, and context window for a specific model.
<code>select_model / set_model</code>	Switch the active model for the current session.
Browser (Playwright)	
<code>browser_navigate</code>	Navigate to a URL.
<code>browser_click</code>	Click an element (supports text/selectors).
<code>browser_type</code>	Type text into an element.
<code>browser_screenshot</code>	Take a full-page or visible screenshot.

Path	Description
	chat interface that connects to <code>manager.py</code> API server.

Development & Testing

File	Purpose
<code>test_summarize.py</code>	Test script for transcript summarization functionality.
<code>requirements.txt</code>	Python dependencies (if present).

How Components Connect



All agents communicate via MCP (Model Context Protocol) using stdio transport. The Manager maintains tool registries from all agents and routes LLM tool calls accordingly.

Plugin API Testing

The Writer's Studio includes a robust Playwright-based test suite specifically designed to test the inter-agent JSON-RPC communication bridge (the `plugin_host.py` MCP Server) without causing unintended side effects (like actually moving emails or editing live documents).

The Test Suite (transcripts/tests/plugin_api_test.spec.ts)

This test suite performs a realistic, integration-level API test by directly spawning the `plugin_host.py` backend and communicating with it via standard input/output (STDIO), exactly as the AI Manager Agent does in production.

It tests all 22+ plugins, including:

- **Email Actions:** `fetch_email`, `read_email`, `flag_spam`, `move_email`
- **Document Actions:** `open_document`, `edit_active_document`
- **Build Automations:** `build_jeff`, `make_epub_jeff`, `synchronize_project`

26

How It Works (The MOCK_PLUGINS Architecture)

To make the tests 100% realistic while remaining completely safe, the system uses a bypass flag architecture:

1. **Spawning the Server:** The Playwright `beforeAll` hook uses Node's `child_process.spawn` to launch `python3 plugin_host.py`, injecting a special environment variable: `MOCK_PLUGINS=true`.

2. **Schema Validation:** When the test sends a JSON-RPC request (e.g., `{"method": "tools/call", "params": {"name": "flag_spam", "arguments": {...}}}`), the `plugin_host` fully processes the request. It parses the plugin's python file, extracts the docstring schema, and maps the JSON arguments to positional command-line arguments.

3. **Safe Interception:** Instead of using `subprocess.run` to execute the underlying AppleScript or Python code (which would modify the user's system), `plugin_host.py` intercepts the call immediately prior to execution because it detects the `MOCK_PLUGINS=true` flag.
4. **Mock Response:** It safely returns a standardized `[MOCK_RESPONSE]` string back through the STDIO pipe.

Why This Matters

This approach ensures we are testing the **actual server code** and **argument parsing logic** rather than relying on tautological

27